

Hotel Ganesh International

Website, Booking, Payment, Google Sheets, Reviews, and Map Guide

This PDF explains the full code setup in simple step-by-step language. It is written so you can understand what happens when a guest books a room, pays online, and receives a verified confirmation.

One-line idea

The website is the front desk, the API folder is the hidden staff room, Razorpay collects money, and Google Sheets stores confirmed bookings.

1. Main Parts of the Project

- Frontend: React pages and components inside src/. This is what visitors see in the browser.
- Backend/API: files inside api/. These run secretly on the server and talk to Razorpay and Google Sheets.
- Environment files: .env stores public frontend settings; api/.env stores private backend secrets.
- External services: Razorpay for payments, Google Sheets for booking records, Google Business/Profile for reviews, Google Maps for location.

2. Important Files

- src/pages/BookingPage.jsx: Main booking form and Razorpay checkout flow.
- api/create-razorpay-order.js: Creates a Razorpay order after validating booking details.
- api/verify-razorpay-payment.js: Verifies Razorpay signature and saves confirmed booking.
- google-sheets-apps-script.js: Code pasted into Google Sheets Apps Script to append rows.
- src/components/GoogleReviews.jsx: Shows review cards and the Write a Review button.
- src/components/FooterMap.jsx: Shows the Google Map using Barh, Patna, Bihar query.
- vite.config.js: Lets API routes work locally during npm run dev.

3. Booking Flow Like a Child

- Guest fills name, phone, email, dates, room type, room count, and guests.
- Browser calculates a visible total so the guest can see the price.
- When guest clicks Pay, the browser sends booking details to the backend.
- Backend checks the details again and calculates the real total itself.
- Backend asks Razorpay to create an order for that real total.
- Razorpay gives back an order ID. The browser opens Razorpay checkout.

Why the backend calculates amount again

A browser can be edited by a clever user. If the website trusted the browser amount, someone could try to pay Rs. 1 for a Rs. 4032 booking. The backend prevents this by calculating the amount again.

4. Razorpay Order Creation

File: api/create-razorpay-order.js

- Loads RAZORPAY_KEY_ID and RAZORPAY_KEY_SECRET from api/.env.
- Checks that the room exists.
- Checks email, phone, dates, guests, and room availability.
- Calculates subtotal, GST, and total.
- Calls Razorpay Orders API and creates an order.
- Returns order ID, amount, currency, receipt, and booking summary to the frontend.

Example total: 1200 x 3 nights = 3600, GST 12% = 432, total = 4032

5. Razorpay Payment Verification

File: api/verify-razorpay-payment.js

- After payment, Razorpay returns order ID, payment ID, and signature.
- Frontend sends those details to the backend.
- Backend creates its own expected signature using the Razorpay secret.
- If Razorpay signature and backend signature match, the payment is real.
- Only then the website shows Booking Confirmed.

Important

A bank message means money came in. Signature verification proves that this website booking belongs to that Razorpay payment.

6. Google Sheets Booking Record

File: google-sheets-apps-script.js

- You pasted this code into Google Sheets > Extensions > Apps Script.
- You deployed it as a Web App and copied the /exec URL.
- That URL is stored in api/.env as GOOGLE_SHEETS_WEBHOOK_URL.
- After payment verification, the backend posts booking details to that URL.
- Google Apps Script appends a row in the Bookings sheet.

Sheet columns

- Confirmed At
- Booking Reference
- Guest Name
- Phone
- Email
- Room

-
- Check In
 - Check Out
 - Nights
 - Rooms
 - Guests
 - Subtotal
 - GST
 - Total Paid
 - Payment ID
 - Order ID

7. Environment Files

api/.env: private backend values

```
RAZORPAY_KEY_ID=test or live xxxxx  
RAZORPAY_KEY_SECRET=your secret key
```

```
GOOGLE_SHEETS_WEBHOOK_URL=https://script.google.com/macros/s/.../exec
```

Never expose RAZORPAY_KEY_SECRET in frontend files. It belongs only on the backend/server.

Root .env: public frontend values

```
VITE_GOOGLE_REVIEW_URL=https://ln.pnqelr/ /review
```

```
VITE_GOOGLE_MAP_QUERY=Hotel Ganesh International, Barh, Patna, Bihar
```

Vite only reads frontend variables from the root .env file. That is why the review link needed to be placed outside api/.env.

8. Google Review Button

File: src/components/GoogleReviews.jsx

- The section shows review cards.
- The Write a Review button uses VITE_GOOGLE_REVIEW_URL.
- A Google Maps API key is not needed just for the Write a Review button.
- Live Google reviews need Google Maps API key and exact Place ID, but that is optional.

9. Google Map Location

File: src/components/FooterMap.jsx

- The map query is set to Hotel Ganesh International, Barh, Patna, Bihar.
- This avoids Google choosing another Hotel Ganesh International in a different state.
- If you use VITE_GOOGLE_PLACE_ID later, make sure it is the exact Barh listing.

10. Local Development Setup

File: vite.config.js

-
- Normally Vite only runs the React frontend.
 - I added local middleware so /api/create-razorpay-order and /api/verify-razorpay-payment work during npm run dev.
 - After changing .env or vite.config.js, restart npm run dev.

npm run dev

11. Speed Improvements

- Large PNG hotel photos were converted to smaller JPG files.
- Images now lazy-load where possible.
- Google Reviews script is delayed until the review section is near the screen.
- Razorpay checkout script is preloaded on the booking page.
- Build output now shows much smaller image sizes, mostly under 80 KB and the largest around 164 KB.

12. Going Live Checklist

- Activate Razorpay Live Mode after KYC/business approval.
- Replace test keys with live keys in Vercel environment variables.
- Add GOOGLE_SHEETS_WEBHOOK_URL to Vercel environment variables.
- Add VITE_GOOGLE_REVIEW_URL and VITE_GOOGLE_MAP_QUERY to Vercel environment variables.
- Deploy again after changing environment variables.
- Make one small live payment test.
- Confirm payment appears in Razorpay Dashboard.
- Confirm booking row appears in Google Sheets.
- Confirm map points to Barh, Patna, Bihar.
- Confirm Write a Review button opens your Google Business review page.

13. Common Problems

Payment says cannot create order

- Check RAZORPAY_KEY_ID and RAZORPAY_KEY_SECRET in api/.env or Vercel env.
- Restart npm run dev after env changes.
- Make sure the backend API route is available.

Payment done but not confirmed

- The server could not verify signature or save to Google Sheets.
- Check Razorpay secret and Google Sheets webhook URL.
- Ask guest for payment ID if needed.

Write a Review button missing

- Put VITE_GOOGLE_REVIEW_URL in root .env, not api/.env.
- Restart npm run dev.

Map shows wrong hotel

- Use VITE_GOOGLE_MAP_QUERY=Hotel Ganesh International, Barh, Patna, Bihar.
- Do not use a wrong Place ID.

14. Final Flow

- Guest fills booking form.
- Backend creates Razorpay order with safe server amount.
- Guest pays with Razorpay.
- Backend verifies Razorpay signature.
- Backend saves confirmed booking to Google Sheets.
- Website shows Booking Confirmed.

Simple memory trick

Frontend shows. Backend protects. Razorpay collects. Google Sheets records.
Google Review link grows reputation. Google Map guides guests.